Debugging Common Errors

How To Read Python Error Messages

When Python finds a problem in your program, it **stops immediately** and prints an error message in the terminal. This message is not there to punish you — it is Python explaining **why it could not continue**.

A Python error message almost always contains the same key parts:

The error type This is the name of the error, such as `SyntaxError`, `NameError`, or `TypeError`. The error type tells you what *kind* of problem Python detected. Start here so you know which section of this document to read.

The traceback Below the error type, Python often prints a traceback. A traceback is a list of lines showing **how Python got to the error**. In beginner programs, this usually only shows one file and a few lines. The **last line of the traceback** is usually the most important, because it points to where Python finally got stuck.

The line number and code line Python will show the line number and often reprint the exact line of code that caused the error. This is Python saying, "I had a problem right here." For beginner code, *the mistake is almost always* **on that line or the line right above it**.

The pointer or highlight Sometimes Python adds a small arrow or caret (`^`) under part of the line. This is Python pointing to the exact character or area where it noticed something wrong. This does not always mean that exact character is the problem, but it is very close.

When debugging, always follow this rule:

1. Read the error type.
2. Go to the line number Python shows.
3. Assume the mistake is small.
4. Fix one thing.
5. Run the program again.

For a full list of possible errors that you might encounter, visit the python documentation that outlines all of the built in error types.

Code Structure (Code Is Invalid Before Running)

`SyntaxError`

What it means: Python cannot understand the way the code is written. This usually means a symbol is missing or in the wrong place.

Check the exact line Python points to

1. Go to the line number shown in the error.
2. Look carefully for missing `)`, `]`, `}`, `:`, or `,`.
3. Make sure strings start and end with the same quote.
4. If the line has `if`, `elif`, `for`, or `while`, make sure it ends with `:`.
5. Ensure that variable names do not have spaces in them.
6. Run the program again.

Check the line above

1. Look at the line directly above the one Python points to.
2. Look for an opening symbol that never closes.
3. Add the missing symbol.
4. Run the program again.

`IndentationError`

What it means: Python expected a line to line up with nearby code, but it doesn't. One line is indented differently than Python expects.

Line up the spacing

1. Go to the line Python points to.
2. Look at the lines directly above it.
3. Make sure this line starts at the same position.
4. If the line above ends with `:`, indent this line once.
5. Run the program again.

`TabError`

What it means: One line starts with different spacing than nearby lines. You likely added or removed a space or tab by accident.

Fix the spacing on the problem line

1. Go to the line number shown in the error.
2. Look only at the very start of that line.
3. Compare it to the lines above and below.
4. Remove or add spaces so it lines up.
5. Run the program again.

Names and Imports (Python Can't Find Something)

`NameError`

What it means: Python does not know the name you are using. This usually means a spelling mistake or using something before it exists.

Make sure the name exists

1. Look at the variable or function causing the name error.
2. Ensure that that variable or name is defined somewhere else in the code.
   - For **functions** this will be the line where the function is defined with the `def` keyword.
   - For **variables** this will be the line where the variable is initialized with a starting value.
3. If the function or variable does not exist anywhere, add it somewhere above the line the error occred.
4. Run the program again.

Check spelling

1. Find the unknown name in the error message.
2. Look for that name in your code.
3. Compare spelling and capitalization.
4. Make all versions match exactly.
5. Run the program again.

Check order

1. Find where the name is first used.
2. Scroll up and look for where it should be created.
3. If it isn't created yet, create it above.
4. Run the program again.

`ImportError`

What it means: Python found the module, but not the specific thing you tried to import from it.

Simplify the import

1. Look at the import line.
2. Change `from module import thing` to `import module`
3. Run the program.
4. If this works, the problem was the `thing` name.
5. Fix the name and try again.

`ModuleNotFoundError`

What it means: Python cannot find the module at all. It is either not installed or Python is using the wrong version.

Assume it isn't installed

1. Look at the module name in the error.
2. Install the module the same way you were shown in class.
3. Run the program again.

Check the Python version

1. Check which Python interpreter your editor is using.
2. Switch to the correct Python version for the project.
3. Run the program again.

`AttributeError`

What it means: You tried to use something that doesn't exist inside an object or module. This often happens when the wrong Python version or missing module is being used.

Check the Python version

1. Check which Python interpreter is selected.
2. Make sure it matches the project you are working on.
3. Run the program again.

Check the module

1. Confirm the module is installed for this Python version.
2. Reinstall it if you are unsure.
3. Run the program again.

Check the name

1. Look at the part after the dot (`object.attribute`).
2. Check spelling and capitalization.
3. Fix it and run the program again.

Types and Values (Data Doesn't Match)

`TypeError`

What it means: Python received the wrong type of data. This often happens when mixing numbers and strings.

Check the data types

1. Find the line Python points to.
2. Identify the values being used together.
3. Decide whether they should be numbers or strings.
4. Convert them using `int()`, `float()`, or `str()`.
5. Run the program again.

`ValueError`

What it means: The type is correct, but the value itself cannot be used. This often happens with user input.

Check the actual value

1. Print the value right before the error line.
2. Run the program and see what the value really is.
3. Fix the input or clean it (for example, remove extra spaces).
4. Run the program again.

Indexes and Keys (Data Doesn't Exist)

`IndexError`

What it means: You tried to access a position that doesn't exist in a list or string.

Check the size

1. Print the list or string.
2. Print its length using `len(...)`.
3. Make sure your index is smaller than the length.
4. Fix the index.
5. Run the program again.

`KeyError`

What it means: You tried to access a dictionary key that doesn't exist.

Check the keys

1. Print the dictionary.
2. Look at the keys that actually exist.
3. Fix spelling or capitalization.
4. Run the program again.

Math Errors (Invalid Math)

`ZeroDivisionError`

What it means: Your code tried to divide by zero.

Prevent zero

1. Find the division line.
2. Identify the value being divided by.
3. Check if it is zero before dividing.
4. Handle that case.
5. Run the program again.

`OverflowError`

What it means: A number became too large.

Find runaway math

1. Locate the line that creates the large number.
2. Print the value before it crashes.
3. Add a limit or stop condition.
4. Run the program again.

Resources and Limits (Program Won't Stop)

`MemoryError`

What it means: Your program tried to store too much data.

Look for growing data

1. Find loops that add to lists or strings.
2. Print the size as the program runs.
3. Add a stopping condition.
4. Run the program again.

`RecursionError`

What it means: A function called itself too many times.

Check the stopping rule

1. Find where the function calls itself.
2. Find the condition that should stop it.
3. Make sure the condition can actually happen.
4. Run the program again.

Debugging rule to remember: If Python points at a line, **start there**. Most beginner bugs are caused by **one character, one space, or one missing line**.

—— Make something worth making ——